Constructor University Bremen

Natural Science Laboratory Communications

Fall Semester 2025

Lab Experiment 3

Author of the report Mr. Nikoloz Burduli

Date of execution : November 10, 2025

1.1. A short explanation of the difference between a usual MATLAB simulation and real time DSP code. Also, briefly explain why buffer-oriented processing is needed and what affects the latency and throughput of an algorithm.

There are several differences between a normal Matlab simulation and code that runs in real time on a DSP. One main difference is related to timing. In Matlab, the program can take as much time as it needs to finish the calculations, because the whole signal is already available from the start. In a real-time DSP system, the code has to keep up with the incoming data as it arrives, so the timing becomes very important. Too much delay from input to output means the system is no longer acting in real time.

Another difference is the way the signal is handled. Matlab usually processes the entire signal in one go, while DSP hardware receives data continuously and must process it bit by bit or block by block. Because the samples don't arrive all at once, buffer oriented processing is required. The signal stream is divided into smaller pieces, each one being processed as it comes in. Sometimes the algorithm also needs part of the previous buffer to deal with the next one, which is why careful buffer management is needed.

The size of the buffer has an influence on both latency and throughput. If the buffer is small, the delay is shorter, because the system does not wait long before starting to process the next part. But this also means the DSP has to switch between buffers more often, which can slow down the overall processing. A larger buffer gives better throughput but increases the delay.

1.2. Diagram showing conceptually how your Delay uses a circular buffer to implement the Delay.

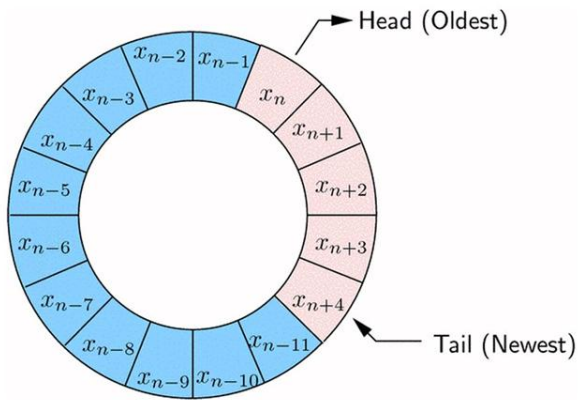

Figure 1. Conceptual diagram of the circular buffer implementation.

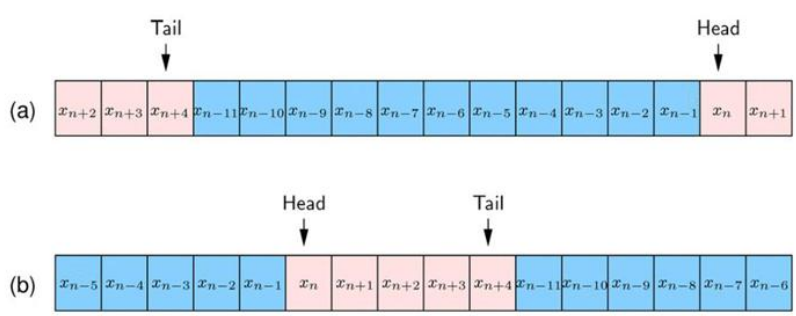

Figure 2. Circular buffer implementation in a processor.

From the diagram in Figure 1 it is clear that the elements in the buffer are arranged in a circular way. New data is added at the back, and when the buffer becomes full, the oldest pieces of data are removed first, following the usual FIFO idea. In an actual processor, the circular buffer is stored in a normal linear memory layout, like in Figure 2, but it still behaves as if it were looping around.

There are two important pointers in the diagram, one showing the head of the buffer and the other pointing to the tail. When a new value arrives, it is written at the tail position and the pointer moves forward. If this new position goes beyond the buffer size, it simply wraps back to the beginning. This way the newest sample replaces the oldest one, keeping the buffer running in a continuous cycle.

1.3. A printout of the MATLAB code that implements your Delay with comments.

```
function [state] = delay_init(Nmax, N)
% [state] = delay_init(Nmax, N);
% Initializes a delay block.
%
% Inputs:
%   Nmax    Maximum delay supported by this block.
%   N       Initial delay
% Outputs:
%   state   State of block
%
% Notes:
%   For this block to operate correctly,
%   you should not pass in more than Nmax
%   samples at a time.

%% 1. Save parameters
state.Nmax = Nmax; % Save max delay
state.N = N;       % Store initial desired delay.

%% 2. Create state variables
% Make the size of the buffer at least twice of the maximum delay.
% Allows us to copy in and then read out in just two steps.
% Also, make it an integer power of 2 for circular addressing.
state.M = 2^(ceil(log2(Nmax))+1);

% Get mask allowing us to wrap index easily
state.Mmask = state.M-1;

% Temporary storage for circular buffer
state.buff = zeros(state.M, 1);

% Set initial head and tail of buffer
% The head starts at 0.
state.n_h = 0;
% The tail is initialized N samples ahead of the head
% to create the initial delay.
state.n_t = N;

end
```

Figure 3. Initialization function for the delay block.

```matlab
function [state_out, y] = delay(state_in, x)
% [state_out, y] = delay(state_in, x);
%
% Delays a signal by the specified number of samples.
%
% Inputs:
%   state_in    Input state
%   x           Input buffer of samples
% Outputs:
%   state_out   Output state
%   y           Output buffer of samples

% Get input state
s = state_in;

% Allocate output buffer
y = zeros(size(x));

% --- Process one sample at a time ---
for ii=0:length(x)-1

    % --- Input Side ---
    % Store the new sample at the tail
    s.buff(s.n_t+1) = x(ii+1);
    % Increment tail index (circularly)
    s.n_t = bitand(s.n_t+1, s.Mmask);

    % --- Output Side ---
    % Get the delayed sample from the head
    y(ii+1) = s.buff(s.n_h+1);
    % Increment head index (circularly)
    s.n_h = bitand(s.n_h+1, s.Mmask);

end

% Output the updated state
state_out = s;
end
```

Figure 4. Processing function for the delay block.

```
% Script to test the delay block. Set up to model the way
% samples would be processed in a DSP program.

% Global parameters
Nb = 10;          % Number of buffers
Ns = 128;         % Samples in each buffer
Nmax = 200;       % Maximum delay
Nd = 10;          % Delay of first block
Nd1 = 16;         % Delay of second block

% Initialize two independent delay blocks
state_delay1 = delay_init(Nmax, Nd);
state_delay2 = delay_init(Nmax, Nd1);

% Generate some random samples.
x = randn(Ns*Nb, 1);
% Reshape into buffers
xb = reshape(x, Ns, Nb);

% Output sample buffers
yb = zeros(Ns, Nb);  % Output of first block
yb1 = zeros(Ns, Nb); % Output of second block

% Process each buffer
for bi=1:Nb
    % Pass buffer through the first delay block
    [state_delay1, yb(:,bi)] = delay(state_delay1, xb(:,bi));
    % Pass the output of the first block into the second
    [state_delay2, yb1(:,bi)] = delay(state_delay2, yb(:,bi));
end

% Convert individual buffers back into a contiguous signal.
y = reshape(yb1, Ns*Nb, 1);
% Total delay
N_total = Nd + Nd1;

% Check if it worked right
n = [0:length(x)-1];

figure(1);
plot(n, x, n, y);
title('Input vs. Delayed Output (Cascaded)');
xlabel('Sample Index');
ylabel('Amplitude');
legend('Input x', 'Output y');

figure(2);
plot(n+N_total, x, n, y, 'x');
title('Input (Shifted) vs. Output (Alignment Check)');
xlabel('Sample Index');
ylabel('Amplitude');
legend('Shifted Input', 'Output');

% Do a check and give a warning if it is not right. Skip first buffer
% in check to avoid initial conditions.
n_chk = 1+[Ns : (Nb-1)*Ns-1];
if any(x(n_chk - N_total) ~= y(n_chk))
    warning('A mismatch was encountered.');
end
```

Figure 5. Scrip to test cascaded delay blocks.

2.1 A diagram showing conceptually how your FIR filter uses a circular buffer to implement the FIR equation.

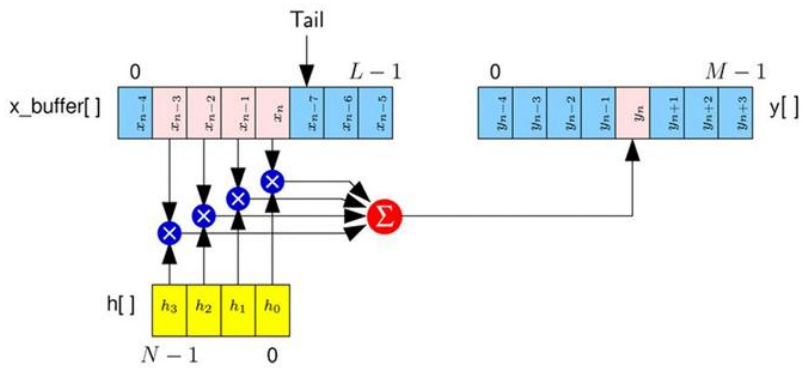

Figure 6. How the FIR Filter uses a circular buffer to implement the FIR equation.

An FIR filter works by using a circular buffer to handle the incoming samples, like the picture shows. The input buffer, called x buffer, has a length L and always keeps the newest sample at the tail, while the older ones get pushed toward the front. The array h holds the impulse response of the filter. Each value in the x buffer is matched with a coefficient in h, and they are multiplied and added together, following the usual convolution idea used in FIR filter design.

The sum of all these multiplications becomes the output for that moment and is stored in the output buffer y. After that, the filter moves on to the next incoming sample and repeats the whole procedure again.

2.2 A printout of the MATLAB code that implements your FIR filter with comments.

```
function [state] = fir_init(h, Ns)
% [state] = fir_init(h, Ns);
%
% Creates a new FIR filter.
%
% Inputs:
%   h        Filter taps
%   Ns       Number of samples processed per block
% Outputs:
%   state    Initial state

%% 1. Save parameters
state.h = h;     % Store filter coefficients
state.Ns = Ns;   % Store the processing block size
```

```
Nh = length(h); % Get number of taps

%% 2. Create state variables
% Make buffer big enough to hold Ns new samples plus Nh-1 old ones
% for the convolution boundary. Make it an integer power of 2.
state.M = 2^(ceil(log2(Ns + Nh)));
% Get mask allowing us to wrap index easily
state.Mmask = state.M-1;

% Temporary storage for circular buffer
state.buff = zeros(state.M, 1); % Initialize buffer to all zeros

% Set initial tail pointer
state.n_t = 0; % Tail starts at 0

end
```

Figure 7. Initialization function for FIR Filter.

```
function [state_out, y] = fir(state_in, x)
% [state_out, y] = fir(state_in, x);
%
% Executes the FIR block.
%
% Inputs:
%   state_in     Input state
%   x            Samples to process (buffer of length Ns)
% Outputs:
%   state_out    Output state
%   y            Processed samples

% Get state
s = state_in;
Ns = length(x);
Nh = length(s.h);
y = zeros(Ns, 1); % Allocate output buffer

% Process one sample at a time from the input buffer
for ii = 0:Ns-1

    % 1. Move the nth sample into the circular buffer (at tail)
    s.buff(s.n_t + 1) = x(ii + 1);

    % 3. Set pointer for convolution.
    % We start at the sample we just added (current tail).
    ptr = s.n_t;

    sumn = 0.0; % Variable to store the sum for the dot product

    % 4. Loop to perform convolution
    for j = 0:Nh-1
        % Multiply sample by corresponding filter coefficient
        sumn = sumn + s.buff(ptr + 1) * s.h(j + 1);

        % Move the pointer backward (circularly) to get the next oldest sample
        % Adding s.M ensures the result is positive before bitand
        ptr = bitand(ptr - 1 + s.M, s.Mmask);
    end

    % 5. Move sum into the output position
    y(ii + 1) = sumn;
```

```

    % 2. Increment tail pointer (done after processing)
    s.n_t = bitand(s.n_t + 1, s.Mmask);

end

% Return updated state
state_out = s;
end
```

Figure 8. Processing function for FIR Filter.

```
% test_fir1.m
%
% Script to test the FIR filter.

% Global parameters
Nb = 100;        % Number of buffers
Ns = 128;        % Samples in each buffer
yb = zeros(Ns, Nb); % initialize the output buffer and fill it with zeroes

% Generate filter coefficients using windowing method
p.beta = 0.5;
p.fs = 0.1;
p.root = 0; % 0=rc, 1=root rc
M = 64;
[h, f, H, Hi] = win_method('rc_filt', p, 0.2, 1, M, 0);
x = randn(Ns*Nb, 1); % Generate some random samples.

% Type of simulation
% stype = 0; % Do simple convolution
stype = 1; % DSP-like filter

% Compute the output for the ideal case using convolution
if stype == 0
    y = conv(x, h);
elseif stype == 1
    % Simulate realistic DSP filter

    % Initialize the FIR filter block
    state_f1 = fir_init(h, Ns);
    % Initialize the output buffer
    yb = zeros(Ns, Nb);
    % Reshape input into buffers
    xb = reshape(x, Ns, Nb);

    % Loop through each buffer, process it, and store the output
    for bi = 1:Nb
      [state_f1, yb(:,bi)] = fir(state_f1, xb(:,bi));
    end

    % Reshape the output buffers back into a single signal
    y = reshape(yb, Ns*Nb, 1);
else
    error('Invalid simulation type.');
end

% Compute approximate transfer function using PSD
Npsd = 200; % Blocksize (# of freq) for PSD
n = 0:length(x)-1;
[Y1, ~] = periodogram(y, [], Npsd, 1);
```

```
[X1, f1] = periodogram(x, [], Npsd, 1);

% Plot ideal H(f) vs. simulated H(f)
figure; % Create new figure
plot(f1, abs(sqrt(Y1./X1)), f, abs(H));
xlim([0 0.2]);
legend('Simulated H(f)', 'Ideal H(f)');
xlabel('Frequency (1/sample)');
ylabel('|H(f)|');
title('FIR Filter Frequency Response');
```

Figure 9. Script to test the FIR filter block.

2.3 Plots showing the ideal response of your filter compared to the simulated response of the filter. Explain any discrepancies.

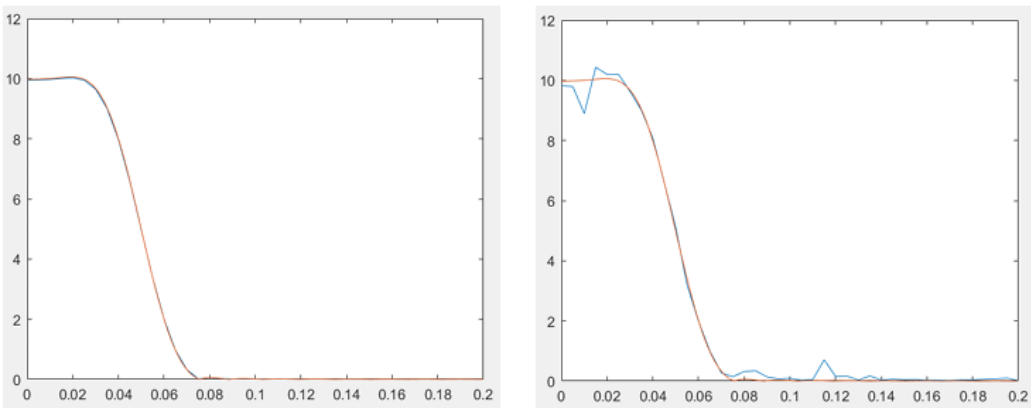

Figure 10. The ideal Response of the filter and The simulated response of the filter.

The ideal response comes from the theoretical design of the filter, so the curve looks almost perfect and only tiny differences can be noticed. The simulated response, on the other hand, is created by running the filter on actual input data, which is randomly generated every time the simulation runs. Even though both results stay close to the theory, the simulated one clearly shows a bit more error. This happens because of the way the filter is implemented in practice, limits in Matlab's numerical precision, and also the windowing effect, which adds some small distortion to the result.

2.4 Any problems you ran into in the lab and how you fixed them

During the lab I ran into a few issues before everything started working properly. One of the first problems showed up when I was testing the Delay function. The head and tail positions of the circular buffer were not initialized correctly, so the samples were being read in the wrong order. After checking the buffer logic again, I fixed the initial values and the delay block started giving the expected output.

Another error appeared in Matlab saying "Unrecognized function or variable 'win_method'." This happened because the required file was not on the Matlab path. I solved it by adding the correct folder manually using the addpath command in the Command Window, and then the script ran without problems.

There were also a couple of mistakes in my code that caused the filter output to behave oddly. With some help from the TA, I went through the lines one by one and corrected the typos and wrong indices.

Besides these, I also had an issue where the plot kept showing zeros for the first few samples. It turned out that I forgot to clear an old buffer from a previous run, so the filter was using outdated values. After resetting all buffers before running the simulation again, the output finally matched the expected shape.